Universidad del Valle de Guatemala

Curso: Programación Orientada a Objetos

Segundo Ciclo, 2026



Ejercicio 4 - Herencia

Nicolas Mercado- 262119

1. Requisitos funcionales

El sistema deberá:

- Registrar un vehículo indicando placa, marca, modelo y tarifa diaria, más el dato particular de su categoría (pasajeros y transmisión en automóviles; cilindraje en motocicletas; capacidad en toneladas en camionetas).
- Impedir el registro de una placa vacía o ya existente en la flota.
- Validar que la tarifa diaria y el dato particular de cada categoría sean mayores que cero.
- Marcar todo vehículo recién registrado como disponible.
- Consultar el catálogo completo de la flota, con sus características y disponibilidad.
- Buscar un vehículo por placa para cotizar un alquiler, indicando la cantidad de días.
- Calcular el costo total de un alquiler según la categoría del vehículo y la cantidad de días.
- Permitir cotizar un vehículo aunque esté ocupado, sin poder alquilarlo mientras no sea devuelto.
- Validar que la cantidad de días sea un entero positivo.
- Confirmar un alquiler solo si el vehículo está disponible, cobrando el monto completo y marcándolo como ocupado.
- Impedir que un mismo vehículo tenga dos alquileres activos simultáneos.
- Permitir cancelar una cotización sin alterar el estado del vehículo ni los ingresos.
- Registrar la devolución de un vehículo identificándolo por placa, y marcarlo nuevamente como disponible.
- Impedir devolver un vehículo que ya está disponible.
- Impedir cotizar, alquilar o devolver un vehículo con placa inexistente.
- Mostrar cuántos vehículos hay registrados, disponibles y alquilados en cada categoría.
- Mostrar el total de ingresos acumulados, contando solo alquileres confirmados.
- Presentar un menú con las operaciones disponibles hasta que el usuario decida salir.
- Validar las entradas para evitar registros y operaciones inconsistentes, sin terminar el programa ante datos con formato incorrecto.
- Mostrar los montos monetarios con dos decimales.

Reglas de cobro

Categoría	Base	Recargo adicional
Automovil	tarifaDiaria * dias	+Q50 por día, solo si transmisionAutomatica es verdadero
Motocicleta	tarifaDiaria * dias	+Q75 único por todo el alquiler, solo si cilindraje > 250
CamionetaCarga	tarifaDiaria * dias	+Q100 * capacidadToneladas * dias (siempre, sin importar la carga real transportada)

Condiciones que permiten o impiden cada operación

Operación	Se permite cuando...	Se rechaza cuando...	Efecto sobre el estado
Registrar vehículo	placa no vacía, no repetida; tarifa y dato particular > 0	placa vacía/duplicada, o algún valor numérico ≤ 0	Se agrega a la flota, disponible
Cotizar	la placa existe; días es entero > 0	placa inexistente, o días ≤ 0 o no numérico	Ninguno
Confirmar alquiler	la placa existe y disponible == true	placa inexistente, o el vehículo ya está ocupado	Se marca ocupado, se suma a ingresosAcumulados
Cancelar cotización	siempre disponible como opción	—	Ninguno
Registrar devolución	la placa existe y disponible == false	placa inexistente, o el vehículo ya estaba disponible	Se marca disponible
Reporte general	siempre disponible	—	Ninguno (solo lectura)

Casos especiales que se deben controlar: no calcular un costo sin antes confirmar que la placa existe; no permitir dos alquileres activos sobre el mismo vehículo; no dejar que una cotización o una cancelación cambien ingresosAcumulados; no aceptar tarifas, pasajeros, cilindrajes o capacidades ≤ 0; no aceptar días ≤ 0 o no enteros; validar texto donde se espera un número sin detener el programa; no permitir placas vacías o repetidas al registrar.

2. Clases propuestas

Clase Vehiculo (abstracta)

Representa las características y el comportamiento comunes a todo vehículo de la flota, sin importar su categoría.

Sus responsabilidades son:

Guardar los datos comunes a todo vehículo (placa, marca, modelo, tarifa diaria).

Controlar su propia disponibilidad.

Definir el contrato para que cada categoría calcule su costo de alquiler.

Clase Automovil (extends Vehiculo)

Representa un automóvil, con los datos que interesan a las familias que lo alquilan.

Sus responsabilidades son:

Guardar la cantidad de pasajeros y el tipo de transmisión.

Calcular su propio costo de alquiler, aplicando el recargo por transmisión automática.

Clase Motocicleta (extends Vehiculo)

Representa una motocicleta.

Sus responsabilidades son:

Guardar el cilindraje.

Calcular su propio costo de alquiler, aplicando el cargo único si supera los 250cc.

Clase CamionetaCarga (extends Vehiculo)

Representa una camioneta de carga.

Sus responsabilidades son:

Guardar la capacidad máxima en toneladas.

Calcular su propio costo de alquiler, aplicando el recargo por tonelada-día.

Clase RentaMovil

Representa la operación del negocio: administra toda la flota y controla los alquileres.

Sus responsabilidades son:

Registrar vehículos nuevos, validando que la placa no esté repetida.

Buscar vehículos por placa.

Cotizar un alquiler sin alterar el estado del vehículo.

Confirmar alquileres, marcando el vehículo como ocupado y acumulando el ingreso.

Registrar devoluciones, marcando el vehículo como disponible.

Generar el reporte general de la flota y los ingresos.

Clase Main

Será la encargada de interactuar con el usuario.

Sus responsabilidades son:

Contener el método main.

Crear la instancia de RentaMovil e inicializarla con vehículos de ejemplo.

Mostrar el menú.

Leer las opciones y los datos ingresados.

Invocar los métodos correspondientes de RentaMovil.

Mostrar los resultados y mensajes de error.

La lógica relacionada con el cobro y el estado de los vehículos debe permanecer principalmente en Vehiculo y RentaMovil; Main se ocupará de la interacción con el usuario.

3. Propiedades de la clase

Atributos de Vehiculo

Atributo	Tipo	Visibilidad	Propósito
placa	String	protected	Identifica de forma única al vehículo dentro de la flota.
marca	String	protected	Almacena la marca del vehículo.
modelo	String	protected	Almacena el modelo del vehículo.
tarifaDiaria	double	protected	Tarifa base que se cobra por cada día de alquiler.
disponible	boolean	protected	Indica si el vehículo puede alquilarse en este momento.

Atributos de Automovil

Atributo	Tipo	Visibilidad	Propósito
pasajeros	int	private	Cantidad de pasajeros que caben en el vehículo.
transmisionAutomatica	boolean	private	Determina si aplica el recargo de Q50 por día.

Atributos de Motocicleta

Atributo	Tipo	Visibilidad	Propósito
cilindraje	int	private	Determina si aplica el cargo adicional de Q75.

Atributos de CamionetaCarga

Atributo	Tipo	Visibilidad	Propósito
capacidadToneladas	double	private	Capacidad máxima de carga; base del recargo por tonelada.

Atributos de RentaMovil

Atributo	Tipo	Visibilidad	Propósito
flota	Map<String, Vehiculo>	private	Guarda todos los vehículos indexados por placa.
ingresosAcumulados	double	private	Acumula el dinero recibido únicamente por alquileres confirmados.

Atributos de Main

La clase Main no necesita atributos de instancia obligatorios. Puede trabajar con variables locales dentro de main.

Métodos de la clase Vehiculo

Método	Retorno	Parámetros	Propósito
Vehiculo(...)	Constructor	String placa, String marca, String modelo, double tarifaDiaria	Inicializa los datos comunes del vehículo y lo deja disponible.
calcularCosto(...)	double *(abstracto)*	int dias	Cada subclase implementa su propia fórmula de cobro.
getPlaca()	String	Ninguno	Devuelve la placa del vehículo.
isDisponible()	boolean	Ninguno	Indica si el vehículo puede alquilarse.
marcarOcupado()	void	Ninguno	Cambia el estado a ocupado al confirmar un alquiler.
marcarDisponible()	void	Ninguno	Cambia el estado a disponible al registrar una devolución.
describir()	String *(abstracto)*	Ninguno	Devuelve las características propias de cada categoría.

Métodos de las subclases de Vehiculo

Método	**Retorno**	**Parámetros**	**Propósito**
Automovil(...)	Constructor	placa, marca, modelo, tarifaDiaria, int pasajeros, boolean transmisionAutomatica	Inicializa los datos del automóvil llamando a super(...).
Motocicleta(...)	Constructor	placa, marca, modelo, tarifaDiaria, int cilindraje	Inicializa los datos de la motocicleta llamando a super(...).
CamionetaCarga(...)	Constructor	placa, marca, modelo, tarifaDiaria, double capacidadToneladas	Inicializa los datos de la camioneta llamando a super(...).
calcularCosto(...)	double	int dias	Reutiliza super.calcularCosto(dia

			s) y suma el recargo propio de la categoría.
describir()	String	Ninguno	Devuelve el dato particular de cada categoría (pasajeros/transmisión, cilindraje o capacidad).

Métodos de la clase RentaMovil

Método	**Retorno**	**Parámetros**	**Propósito**
registrarVehiculo(...)	boolean	Vehiculo v	Agrega el vehículo a la flota si su placa no existe ya.
buscarPorPlaca(...)	Vehiculo	String placa	Devuelve el vehículo correspondiente o null si no existe.
cotizar(...)	String	String placa, int dias	Calcula y devuelve el costo y las características sin modificar el estado.
confirmarAlquiler(...)	boolean	String placa, int dias	Marca el vehículo ocupado y acumula el ingreso si estaba disponible.
registrarDevolucion(...)	boolean	String placa	Marca el vehículo disponible si estaba ocupado.
reporteGeneral()	String	Ninguno	Muestra los conteos por categoría, la disponibilidad y los ingresos acumulados.
esDiasValido(...)	boolean	int dias	Comprueba que la cantidad de días sea un entero positivo.

No es recomendable crear un método que devuelva directamente la colección flota, porque permitiría modificarla desde fuera de la clase y rompería el encapsulamiento.

Métodos de la clase Main

Método	Retorno	Parámetros	Propósito
main(...)	void	String[] args	Inicia el programa, crea la RentaMovil y controla el menú.
inicializarFlota(...)	void	RentaMovil r	Registra al menos dos vehículos de cada categoría al arrancar.
mostrarMenu()	void	Ninguno	Muestra las opciones disponibles.
leerEntero(...)	int	Scanner scanner, String mensaje	Lee y valida entradas numéricas sin permitir que el programa falle.
leerDecimal(...)	double	Scanner scanner, String mensaje	Lee y valida entradas decimales sin permitir que el programa falle.

Los métodos auxiliares de Main pueden ser private static, ya que solamente serán utilizados por el driver program.

4. Valores iniciales de los objetos

Al registrar un vehículo, Main lee los datos por teclado y llama al constructor de la subclase correspondiente (Automovil, Motocicleta o CamionetaCarga). Ese constructor llama a super(placa, marca, modelo, tarifaDiaria), que inicializa los atributos comunes de Vehiculo y deja disponible = true por defecto — nunca hace falta pasarlo como parámetro, porque todo vehículo recién incorporado queda disponible. Luego el constructor de la subclase inicializa su propio atributo (pasajeros+transmisionAutomatica, cilindraje o capacidadToneladas) con el valor recibido.

RentaMovil se inicializa con flota vacío (new HashMap<>()) e ingresosAcumulados = 0.0; ningún vehículo existe hasta que Main llama a registrarVehiculo(...) para cada uno al arrancar el programa.